Clinic Database Management System Project Report

Under the supervision of: Eng. Mohamed ElSayeh & Eng. Habiba Mohamed.
Course: CBIO204 – Fundamentals of Databases.

Student Name:

Youssef Amgad ID:241001061
Mariam Fahmy ID: 241002353
Wafaa Ahmed ID: 241001920
Ahmed Nabil ID: 241001469
Aya Adly ID: 241000377

1. Introduction:

This project involves the design of a database for a Clinic Management System. The system is designed to facilitate the scheduling of appointments, management of medical departments, and storage of patient medical history. It serves as a centralized platform for clinics and doctors to efficiently manage healthcare delivery.

2. Conceptual design:

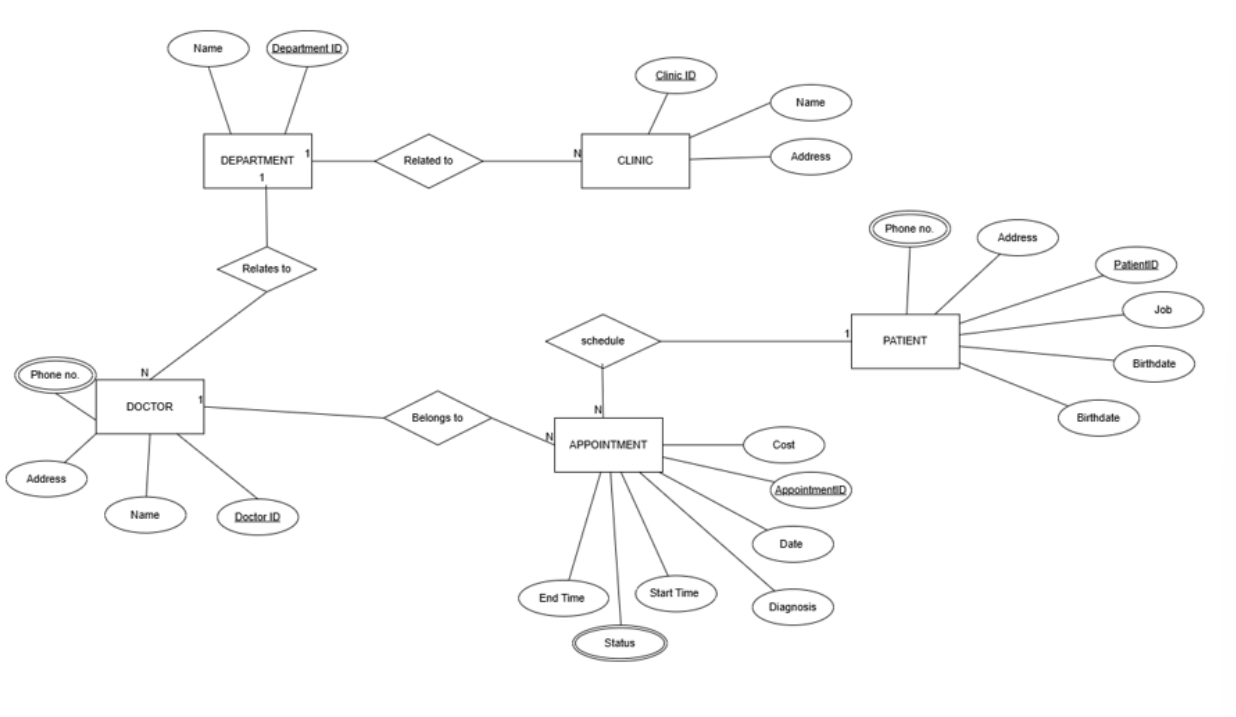
The conceptual design identifies the primary entities and their relationships within the clinic ecosystem. Entities include Departments, Clinics, Doctors, Patients, and Appointments.

2.1 Entity Descriptions:

- Department: Represents medical divisions such as Cardiology or Pediatrics.
- Clinic: Physical locations where patients receive treatment, linked to departments.
- Doctor: Medical professionals assigned to specific departments.
- Patient: Individuals seeking medical services, with records of their demographics and jobs.
- Appointment: The central transaction linking patients, doctors, and clinics, including dates, costs, and diagnosis.

2.2 Attributes for Each Entity:

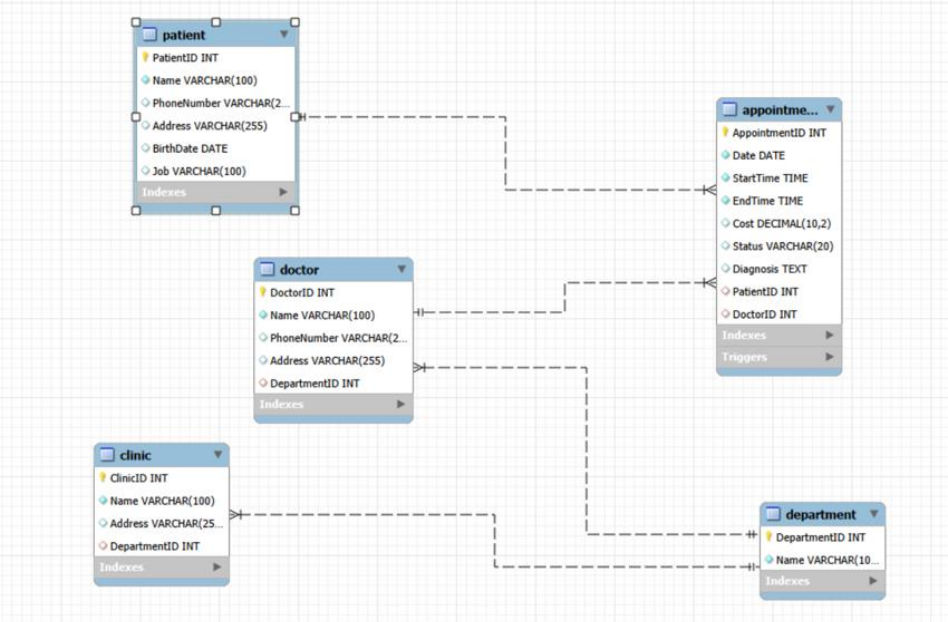
- Department: DeptID, Name
- Clinic: ClinicID, Name, Address
- Doctor: DoctorID, Name, PhoneNumber, Address
- Patient: PatientID, Name, PhoneNumber, Address, BirthDate, Job
- Appointment: AppID, Date, StartTime, EndTime, Cost, Status, Diagnosis



3. ER-Diagram Details:

The ER-Diagram follows the course notation and includes the following constraints:
Cardinality Ratios: Department to Clinic (1:N), Doctor to Appointment (1:N), Patient to Appointment (1:N).
Participation Constraints: Clinic has Total Participation in Department.
Doctor has Total Participation in Department.
Appointment has Total Participation in Doctor and in Patient.

Department has Partial Participation in Clinic and Doctor.
Patient has Partial Participation in Appointment.



4. Relational Database Schema:

Department: DeptID (PK), Name
Clinic: ClinicID (PK), Name, Address, DeptID (FK)
Doctor: DoctorID (PK), Name, PhoneNumber, Address, DeptID (FK)
Patient: PatientID (PK), Name, PhoneNumber, Address, BirthDate, Job
Appointment: AppID (PK), Date, StartTime, EndTime, Cost, Status, Diagnosis, PatientID (FK), DoctorID (FK)

5. Implementation Summary:

Summary of implemented tables and data volume:

Entity Table	Records	Primary Key
Department	10	DeptID
Clinic	10	ClinicID
Doctor	10	DoctorID
Patient	10	PatientID
Appointment	10	AppID

6. DDL & DML:

6.1. Database Creation:

```
CREATE SCHEMA ClinicSystem;
```

```
USE ClinicSystem;
```

The ClinicSystem database was created to manage information related to departments, clinics, doctors, patients, and appointments in a clinic.

6.2. DDL (Data Definition Language):

- **Department Table:**

```
CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);
```

Purpose: Stores all medical departments in the clinic system and each department has a unique DepartmentID.

- **Clinic Table:**

```
CREATE TABLE Clinic (
    ClinicID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Address VARCHAR(255),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

Purpose: Stores clinic branches and their related departments. Foreign key so that every clinic belongs to a valid department.

- **Doctor Table:**

```
CREATE TABLE Doctor (
    DoctorID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    PhoneNumber VARCHAR(20),
    Address VARCHAR(255),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

Purpose: Stores doctor information. Each doctor is connected to one department.

- **Patient Table:**

```
CREATE TABLE Patient (
    PatientID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    PhoneNumber VARCHAR(20),
    Address VARCHAR(255),
    BirthDate DATE,
    Job VARCHAR(100)
);
```

Purpose: Stores patient personal information.

- **Appointment Table**

```
CREATE TABLE Appointment (
    AppointmentID INT PRIMARY KEY,
    Date DATE NOT NULL,
    StartTime TIME NOT NULL,
    EndTime TIME NOT NULL,
    Cost DECIMAL(10, 2),
    Status VARCHAR(20),
    Diagnosis TEXT,
    PatientID INT,
    DoctorID INT,
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);
```

Purpose: Stores appointment details between patients and doctors. Foreign keys maintain relationships with Patient and Doctor tables.

6.3. DML (Data Manipulation Language):

6.3.1. INSERT Statements:

- **Department Data:**

```
INSERT INTO Department (DepartmentID, Name) VALUES
(1, 'Cardiology'),
(2, 'Pediatrics'),
(3, 'Dermatology'),
(4, 'Neurology'),
(5, 'Orthopedics'),
(6, 'Ophthalmology'),
(7, 'Gastroenterology'),
(8, 'Psychiatry'),
(9, 'Endocrinology'),
(10, 'General Medicine');
```

Purpose: Adds medical departments into the Department table such as Cardiology, Pediatrics, and Neurology. These records are used to organize doctors and clinics by specialization.

- **Doctor Data:**
  INSERT INTO Doctor (DoctorID, Name, PhoneNumber, Address, DepartmentID) VALUES
  (101, 'Dr. Alice Smith', '555-0101', '123 Maple St', 1),
  (102, 'Dr. Bob Johnson', '555-0102', '456 Oak Ave', 2),
  (103, 'Dr. Charlie Brown', '555-0103', '789 Pine Rd', 3),
  (104, 'Dr. Diana Prince', '555-0104', '321 Elm Blvd', 4),
  (105, 'Dr. Edward Norton', '555-0105', '654 Cedar Ln', 5),
  (106, 'Dr. Fiona Gallagher', '555-0106', '987 Birch Dr', 6),
  (107, 'Dr. George Miller', '555-0107', '159 Walnut Ct', 7),
  (108, 'Dr. Hannah Abbott', '555-0108', '753 Willow Way', 8),
  (109, 'Dr. Ian Wright', '555-0109', '852 Aspen Dr', 9),
  (110, 'Dr. Jane Foster', '555-0110', '951 Cherry St', 10);

Purpose: Adds doctor records into the Doctor table including names, phone numbers, addresses, and department assignments.

- **Patient Data:**
  INSERT INTO Patient (PatientID, Name, PhoneNumber, Address, BirthDate, Job) VALUES
  (12527, 'John Doe', '555-2001', '101 Apple St', '1985-05-12', 'Software Engineer'),
  (2002, 'Mary Poppins', '555-2002', '202 Cherry Ln', '1970-11-23', 'Teacher'),
  (2003, 'Bruce Wayne', '555-2003', '1007 Wayne Manor', '1980-02-19', 'CEO'),
  (2004, 'Clark Kent', '555-2004', '344 Clinton St', '1978-06-18', 'Journalist'),
  (2005, 'Peter Parker', '555-2005', '20 Ingram St', '2001-08-10', 'Photographer'),
  (2006, 'Diana Prince', '555-2006', 'Gateway City', '1984-03-05', 'Antiquities Dealer'),
  (2007, 'Tony Stark', '555-2007', '10880 Malibu Pt', '1970-05-29', 'Inventor'),
  (2008, 'Steve Rogers', '555-2008', '569 Lefferts Ave', '1918-07-04', 'Artist'),
  (2009, 'Natasha Romanoff', '555-2009', 'Classified', '1984-11-22', 'Special Agent'),
  (2010, 'Wanda Maximoff', '555-2010', '616 Hex Dr', '1989-02-10', 'None'),
  (2011, 'Stephen Strange', '555-2011', '177A Bleecker St', '1930-11-18', 'Neurosurgeon');

Purpose: Adds patient information into the Patient table including personal details such as birth date, address, and occupation.

- **Appointment Data:**
  INSERT INTO Appointment (AppointmentID, Date, StartTime, EndTime, Cost, Status, Diagnosis, PatientID, DoctorID) VALUES
  (501, '2026-05-01', '09:00:00', '09:30:00', 150.00, 'Scheduled', 'Routine Checkup', 12527, 101),
  (502, '2026-05-02', '10:00:00', '10:45:00', 200.00, 'In Progress', 'High Blood Pressure', 2002, 101),
  (503, '2025-12-15', '11:00:00', '11:30:00', 350.00, 'Postponed', 'Joint Pain', 2003, 105),
  (504, '2026-05-10', '14:00:00', '14:30:00', 120.00, 'Scheduled', 'Flu Symptoms', 2004, 110),
  (505, '2024-03-20', '08:30:00', '09:00:00', 500.00, 'Scheduled', 'Chest Pain', 12527, 101),
  (506, '2026-05-11', '13:00:00', '13:30:00', 95.00, 'Scheduled', 'Skin Rash', 2005, 103),

(507, '2026-06-05', '15:30:00', '16:00:00', 250.00, 'Scheduled', 'Anxiety', 2008, 108),
(508, '2023-01-10', '09:00:00', '10:00:00', 1000.00, 'Scheduled', 'Surgery Consult', 12527, 105),
(509, '2026-05-12', '10:30:00', '11:00:00', 180.00, 'Scheduled', 'Eye Exam', 2006, 106),
(510, '2026-05-13', '12:00:00', '12:45:00', 220.00, 'Scheduled', 'Diabetes Follow-up', 2009, 109),
(511, '2026-05-14', '09:00:00', '09:30:00', 300.00, 'Scheduled', 'Migraine', 2011, 104);

Purpose: Adds appointment records into the Appointment table. Each appointment connects a patient with a doctor and stores information such as appointment date, time, diagnosis, cost, and status.

- **Clinic Data:**
  INSERT INTO Clinic (ClinicID, Name, Address, DepartmentID) VALUES
  (10, 'Heart Center A', '123 Maple St, Suite 1', 1),
  (11, 'Kids Care Clinic', '456 Oak Ave, Suite 2', 2),
  (12, 'Skin Health Specialists', '789 Pine Rd, Suite 1', 3),
  (13, 'Neuro Research Unit', '321 Elm Blvd, Wing B', 4),
  (14, 'Joint & Bone Clinic', '654 Cedar Ln, Floor 1', 5),
  (15, 'Vision Care Center', '987 Birch Dr, Suite 5', 6),
  (16, 'Digestive Health Plus', '159 Walnut Ct, Suite 3', 7),
  (17, 'Wellness Mind Clinic', '753 Willow Way, Suite 10', 8),
  (18, 'Hormone Specialist Group', '852 Aspen Dr, Floor 2', 9),
  (19, 'City General Clinic', '951 Cherry St, Wing A', 10),
  (20, 'Cardiology Annex', '123 Maple St, Suite 2', 1);

Purpose: Adds clinic branch information into the Clinic table including clinic names, addresses, and their related departments.

6.3.2. CREATE VIEW:

```
CREATE VIEW vw_Doctor_Schedule_NextMonth AS
SELECT d.Name AS Doctor_Name, COUNT(a.AppointmentID) AS Appt_Count
FROM Doctor d
LEFT JOIN Appointment a ON d.DoctorID = a.DoctorID
WHERE a.Date BETWEEN '2026-06-01' AND '2026-06-30'
GROUP BY d.DoctorID, d.Name;
```

Purpose: Displays the number of appointments each doctor has during the next month.

6.3.3. UPDATE Statements:

```
UPDATE Appointment
SET Status = 'In Progress'
```

```
WHERE AppointmentID = 502;


UPDATE Appointment
SET Cost = Cost * 1.10
WHERE DoctorID IN (SELECT DoctorID FROM Doctor WHERE DepartmentID = 1);
```

Purpose: Updates appointment status. Increases costs for appointments in the Cardiology department.

6.3.4. DELETE Statement:

```
DELETE FROM Patient WHERE PatientID = 2010;
```

Purpose: Removes a patient record from the database.

7. Test Data:

The database contains at least 10 rows in every major table:

AppointmentID	Date	StartTime	EndTime	Cost	Status	Diagnosis	PatientID	DoctorID
501	2026-05-01	09:00:00	09:30:00	150.00	Scheduled	Routine Checkup	12527	101
502	2026-05-02	10:00:00	10:45:00	200.00	In Progress	High Blood Pressure	2002	101
503	2025-12-15	11:00:00	11:30:00	350.00	Postponed	Joint Pain	2003	105
504	2026-05-10	14:00:00	14:30:00	120.00	Scheduled	Flu Symptoms	2004	110
505	2024-03-20	08:30:00	09:00:00	500.00	Scheduled	Chest Pain	12527	101
506	2026-05-11	13:00:00	13:30:00	95.00	Scheduled	Skin Rash	2005	103
507	2026-06-05	15:30:00	16:00:00	250.00	Scheduled	Anxiety	2008	108
508	2023-01-10	09:00:00	10:00:00	1000.00	Scheduled	Surgery Consult	12527	105
509	2026-05-12	10:30:00	11:00:00	180.00	Scheduled	Eye Exam	2006	106
510	2026-05-13	12:00:00	12:45:00	220.00	Scheduled	Diabetes Follow-up	2009	109
511	2026-05-14	09:00:00	09:30:00	300.00	Scheduled	Migraine	2011	104

ClinicID	Name	Address	DepartmentID
10	Heart Center A	123 Maple St, Suite 1	1
11	Kids Care Clinic	456 Oak Ave, Suite 2	2
12	Skin Health Specialists	789 Pine Rd, Suite 1	3
13	Neuro Research Unit	321 Elm Blvd, Wing B	4
14	Joint & Bone Clinic	654 Cedar Ln, Floor 1	5
15	Vision Care Center	987 Birch Dr, Suite 5	6
16	Digestive Health Plus	159 Walnut Ct, Suite 3	7
17	Wellness Mind Clinic	753 Willow Way, Suite 10	8
18	Hormone Specialist Group	852 Aspen Dr, Floor 2	9
19	City General Clinic	951 Cherry St, Wing A	10
20	Cardiology Annex	123 Maple St, Suite 2	1

DepartmentID	Name
1	Cardiology
2	Pediatrics
3	Dermatology
4	Neurology
5	Orthopedics
6	Ophthalmology
7	Gastroenterology
8	Psychiatry
9	Endocrinology
10	General Medicine

DoctorID	Name	PhoneNumber	Address	DepartmentID
101	Dr. Alice Smith	555-0101	123 Maple St	1
102	Dr. Bob Johnson	555-0102	456 Oak Ave	2
103	Dr. Charlie Brown	555-0103	789 Pine Rd	3
104	Dr. Diana Prince	555-0104	321 Elm Blvd	4
105	Dr. Edward Norton	555-0105	654 Cedar Ln	5
106	Dr. Fiona Gallagher	555-0106	987 Birch Dr	6
107	Dr. George Miller	555-0107	159 Walnut Ct	7
108	Dr. Hannah Abbott	555-0108	753 Willow Way	8
109	Dr. Ian Wright	555-0109	852 Aspen Dr	9
110	Dr. Jane Foster	555-0110	951 Cherry St	10

PatientID	Name	PhoneNumber	Address	BirthDate	Job
2002	Mary Poppins	555-2002	202 Cherry Ln	1970-11-23	Teacher
2003	Bruce Wayne	555-2003	1007 Wayne Manor	1980-02-19	CEO
2004	Clark Kent	555-2004	344 Clinton St	1978-06-18	Journalist
2005	Peter Parker	555-2005	20 Ingram St	2001-08-10	Photographer
2006	Diana Prince	555-2006	Gateway City	1984-03-05	Antiquities Dealer
2007	Tony Stark	555-2007	10880 Malibu Pt	1970-05-29	Inventor
2008	Steve Rogers	555-2008	569 Lefferts Ave	1918-07-04	Artist
2009	Natasha Romanoff	555-2009	Classified	1984-11-22	Special Agent
2010	Wanda Maximoff	555-2010	616 Hex Dr	1989-02-10	None
2011	Stephen Strange	555-2011	177A Bleecker St	1930-11-18	Neurosurgeon
12527	John Doe	555-2001	101 Apple St	1985-05-12	Software Engineer

- Represent different medical departments.
- Include multiple doctors and patients.
- Simulate realistic clinic appointments.
- Support testing for joins, aggregation, views, updates, and triggers.

8. Design Justification:

8.1. Database Design Choices:

- **Primary Keys:**

Each table uses a primary key:

- DeptID
- ClinicID
- DoctorID
- PatientID
- AppID

Primary keys maintain every record's uniqueness.

- **Foreign Keys:**

Foreign keys were used to create relationships between tables. Such as:

- DepartmentID in Doctor references Department.
- PatientID and DoctorID in Appointment reference Patient and Doctor.

This prevents invalid data entry.

- **Appointment Table:**

The Appointment table connects doctors and patients together.

This design allows one patient to have many appointments and one doctor to handle many appointments.

- **Views:**

The view vw_Doctor_Schedule_NextMonth was created to simplify schedule reporting.

Benefits: Easier data retrieval, reduced query repetition and better readability.

9. Query Implementation:

- **Patient Names and Jobs Query:**

SELECT Name, Job FROM Patient;

Retrieves all patient names and occupations from the Patient table.

- **Cardiology Clinics Query:**

```
SELECT Name, Address
FROM Clinic
WHERE DepartmentID =
(SELECT DepartmentID
```

```
FROM Department
WHERE Name = 'Cardiology');
```

This query uses a subquery to locate the DepartmentID of Cardiology, then retrieves all clinics assigned to that department.

- **LIKE Operator Query:**

Update statement to modify the appointment data:

```
UPDATE Appointment
SET Diagnosis = 'Patient has fatty liver symptoms',
Date = '2026-01-10'
WHERE AppointmentID = 501;
```

This updates the diagnosis and date for appointment 501.

```
SELECT DISTINCT p.Name
FROM Patient p
JOIN Appointment a ON p.PatientID = a.PatientID
WHERE a.Diagnosis LIKE '%fatty liver%'
AND a.Date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR);
```

Searches for patients diagnosed with fatty liver within the last year

The LIKE operator for the partial matching of text inside the Diagnosis column.

- **INNER JOIN Query:**

```
SELECT Doctor.Name AS Doctor_Name,
Department.Name AS Dept_Name
FROM Doctor
INNER JOIN Department
ON Doctor.DepartmentID = Department.DepartmentID;
```

This query combines the Doctor and Department tables to display each doctor with their department name.

- **LEFT JOIN Query:**

```
SELECT p.Name, a.Date
FROM Patient p
LEFT JOIN Appointment a
ON p.PatientID = a.PatientID;
```

This query displays all patients, including patients who do not currently have appointments.

- **Aggregation Query:**

```
SELECT SUM(Cost) AS Total_Paid
FROM Appointment
WHERE PatientID = 12527
AND Date >= DATE_SUB(CURDATE(), INTERVAL 3 YEAR);
```

Calculates the total amount paid by patient 12527 over the last three years.

- **GROUP BY and HAVING Query:**

```
SELECT d.Name, AVG(a.Cost) AS Avg_Cost
FROM Department d
JOIN Doctor doc ON d.DepartmentID = doc.DepartmentID
JOIN Appointment a ON doc.DoctorID = a.DoctorID
GROUP BY d.Name
HAVING AVG(a.Cost) > 100;
```

Calculates the average appointment cost for each department and filters departments with averages greater than 100.

- **VIEW Creation:**

```
CREATE VIEW vw_Doctor_Schedule_NextMonth AS
SELECT d.Name AS Doctor_Name,
COUNT(a.AppointmentID) AS Appt_Count
FROM Doctor d
LEFT JOIN Appointment a
ON d.DoctorID = a.DoctorID
WHERE a.Date BETWEEN '2026-06-01' AND '2026-06-30'
GROUP BY d.DoctorID, d.Name;
```

This view summarizes the number of appointments assigned to each doctor during June 2026.

```
SELECT * FROM vw_Doctor_Schedule_NextMonth;
```

Retrieves data from the view

- **UPDATE Query:**

```
UPDATE Appointment
SET Status = 'In Progress'
WHERE AppointmentID = 502;
```

This updates the appointment status for appointment 502.

- **DELETE Query:**

```
DELETE FROM Patient
```

WHERE PatientID = 2010;

This deletes the patient record with ID 2010.

10. Triggers and Their Purpose:

- **prevent_doctor_overlap Trigger:**

Purpose:

- Prevents two appointments from overlapping for the same doctor.
- Doctors cannot be double-booked.

The trigger runs BEFORE INSERT so invalid appointments are blocked before entering the database.

The overlap condition checks:

(NewStart < ExistingEnd) AND (NewEnd > ExistingStart)

If this condition is true, the appointment times overlap.

COUNT(*) checks how many conflicting appointments already exist.

SIGNAL SQLSTATE generates a custom error message and prevents insertion.

- **apply_appointment_tax Trigger:**

Purpose:

- Automatically adds a 10% tax to appointment costs before insertion.
- Billing consistency across appointments.
- If a cost value exists, SQL multiplies the value by 1.10.
- All stored costs automatically include tax without requiring manual calculation.

- **validate_appointment_status Trigger:**

Purpose:

- Restricts appointment status values to approved options only.
- Prevents invalid or inconsistent status entries.

The trigger validates status values before an update occurs.
NOT IN checks whether the new status matches approved values.

If an invalid value is entered, SQL blocks the update and displays an error message.

Allowed values: scheduled, in progress, or postponed.

The trigger checks lowercase values only. If a value such as “Scheduled” or “In Progress” is inserted with uppercase letters, the trigger may reject it.

11. GUI:

Summary: This OS is a full stack medical registry system designed to optimize clinic management workflows, track patient diagnostics, organize departmental staff directories, and manage active scheduling constraints. Built on a modular three-tier software architecture, the application decouples frontend data presentation from backend application logic and data persistence layers.

The system uses an integrated Client-Server-Database software pattern to build a secure loop for dynamic data transmission. The individual tiers function as follows:

- **The Client layer (Front end UI):** Responsible for translating raw payloads into a clean and easy UI. It used website composed of HTML documents that were paired with CSS layouts for visual ease.
- **The APP layer (Backend):** It was written in Python as we are most familiar with Programming Language using a Flask Framework. The server handles primary operational tasks: it opens web route interfaces, catches form structures from client views.
- **The Persistence Layer (Relational Database):** This was constructed inside MySQL the storage system models physical clinic operations across five interlocked relational schema arrays: patient profiles, specialized doctor assignments, clinic venues, operational departments, and transactional appointment tables.

Core Technical Features & Data Flows.

- Real-Time Dashboard Analytics: Upon loading the application gateway, the backend executes targeted structural queries using SQL aggregate statements. The system calls descriptive counters to compute totals across live patient directories, doctor staff rolls, and active diagnostic queues. These results populate dynamic status tiles on the administrator interface immediately upon execution.

- Parametric Patient Registry Search: The directory view features an integrated filter matrix that allows real-time lookups. When an administrator types into the system search interface, the backend maps the parameter string and inserts it into a SQL search query using conditional LIKE modifiers. This allows a user to look up a patient record by full matching name, matching text substrings, or exact numeric identifier keys using a single, unified search input field.

- Transactional State Modification: When updating status fields, the interface packages the selected identifiers into an HTTP POST request payload. The Python core accepts the payload arguments, initializes a targeted transaction cursor, applies the changes using secure UPDATE statements, and performs an explicit commit operation to guarantee permanent drive storage.

12. Questions:

Question 1: How would you prevent two appointments for the same doctor from overlapping?

A before insert trigger runs before each new row is saved and can raise an error to cancel the insert.

```
DELIMITER //

CREATE TRIGGER prevent_doctor_overlap
BEFORE INSERT ON Appointment
FOR EACH ROW
BEGIN
   DECLARE overlap_count INT;


   -- Check if any existing appointment for the same doctor and date overlaps
   -- Overlap Condition: (NewStart < ExistingEnd) AND (NewEnd > ExistingStart)
   SELECT COUNT(*) INTO overlap_count
   FROM Appointment
   WHERE DoctorID = NEW.DoctorID
     AND Date = NEW.Date
     AND NEW.StartTime < EndTime
     AND NEW.EndTime > StartTime;


   -- If an overlap exists, block the insertion and return an error message
   IF overlap_count > 0 THEN
      SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Error: Doctor is already booked during this time slot.';
    END IF;
END; //
```

Question 2: Describe a query to fetch each patient's name plus their most recent appointment date

This requires a left join between Patient and Appointment so patients with no appointments are still included. Then max(a.Date) as an aggregate to find the latest appointment, and group by the patient's ID and name to combine multiple rows into one per patient.

```
SELECT
    p.PatientID,
    p.Name AS Patient_Name,
    MAX(a.Date)    AS Most_Recent_Appointment
FROM Patient p
LEFT JOIN Appointment a ON p.PatientID = a.PatientID
GROUP BY p.PatientID, p.Name
ORDER BY Most_Recent_Appointment DESC;
```

Question 3: A view showing each doctor's upcoming appointment count for next month. What would its GROUP BY look like?

The view joins Doctor with Appointment and filters for dates within the next month. Group by can be used to show one row per doctor, with a count of their appointments. Group by DoctorID (the unique key) and Name (to display it), then COUNT() to count appointments for each group.

```
CREATE VIEW vw_Doctor_Schedule_NextMonth AS
SELECT
    d.DoctorID,
    d.Name AS Doctor_Name,
    COUNT(a.AppointmentID) AS Appointment_Count
FROM Doctor d
LEFT JOIN Appointment a
       ON d.DoctorID = a.DoctorID
      AND a.Date BETWEEN '2026-06-01' AND '2026-06-30'
GROUP BY d.DoctorID, d.Name;
```